List 2: Object-oriented programming in Python

compiled by Adam Oleszko, Witold Dyrka

19th October 2026

General information

All code should be written according to The Style Guide for Python Code. Each function or class should be described with a docstring (see Docstring Conventions). We recommend sticking to the Google or NumPy documentation comment styles.

If pre- or postconditions or *invariants* are not met, the code should raise an *exception*.

Both the instantiation and the functionality of each class should be thoroughly tested. The tests should cover a diverse range of scenarios to ensure comprehensive and reliable code performance.

Carefully avoid plagiarism. Each external source of code should be precisely referenced. The use of prompt-based artificial intelligence tools to support completing problem sets is permitted provided that *full* documentation of the code development process is submitted, including specifying all tools and models used, their versions, and all prompts. The use of plugin-based coding assistants is prohibited.

Exercise 0

No bonus points are associated with completing this exercise;-(

Read chapters 15 to 18 from the book *Think Python: How to Think Like a Computer Scientist* by Allen B. Downey (Green Tea Press, 2015). They provide a gentle but sound introduction to object-oriented programming (OOP) using Python. You can also use other Python OOP books of your choice.

Exercise 1

Create class `Solution` to represent a chemical or biochemical solution of a given substance. The class should include:

- attribute `name`,
- attribute `concentration` (in mg/mL),
- attribute `volume` (in mL).

Implement the following methods and operators:

- `__init__()` – constructor validating that the concentration > 0 and volume > 0,
- `__str__()` – returns formatted information about the solution,
- `add(other)` – combines two solutions of the same solute, updating the concentration and total volume,
- `dilute(factor)` – reduces concentration by the given dilution factor,
- `__eq__()` – returns `True` if two solutions have the same name and concentration.

Write code demonstrating the functionality of the class and verify its behavior.

Exercise 2

Create two classes, `Patient` and `Hospital`, to represent a simple hospital management system. In class `Patient`:

- attribute `name`,
- attribute `dob` – date of birth (string or date),
- attribute `id` – unique patient identifier,
- attribute `diagnosis` – diagnosis name (or code according to ICD-10),
- attribute `admitted` – a boolean indicating whether the patient is currently admitted.

Implement the following methods:

- `__init__()` – constructor initializing the attributes,
- `admit()` – sets admitted to `True`,
- `discharge()` – sets admitted to `False`,
- `__str__()` – returns a string summarizing the patient's data.

In class `Hospital`:

- attribute `name`,
- attribute `patients` – list of `Patient` objects.

Implement the following methods:

- `__init__()` – constructor initializing the hospital name and empty patient list,
- `add_patient(patient)` – adds a Patient object to the list,
- `discharge_patient(id)` – marks a patient as discharged by their ID,
- `get_patient_list()` – returns IDs of all currently admitted patients.

Write code demonstrating the functionality of both classes.

Exercise 3

Create three classes: `DNASequence`, `RNASequence`, `ProteinSequence`. Below are the attributes and methods listed for each class.

In `DNASequence`:

- attribute `identifier`,
- attribute `data` – actual sequence,
- attribute `valid_chars` – set of valid characters to be used to verify the correctness of sequence data,
- method `__len__()` – returns the length of the sequence data,
- method `__str__()` – returns representation in the FASTA-like format:

```
>identifier
data
```

- method `mutate(position, value)` – modifies sequence data at the given `position` to the specified `value`,
- method `find_motif(motif)` – returns the positions of the given `motif` in the sequence,
- method `complement()` – returns the complementary sequence,
- method `transcribe()` — returns a `RNASequence` instance representing the result of the transcription of the sequence.

In `RNASequence`:

- attribute `identifier`,
- attribute `data` — actual sequence,
- attribute `valid_chars` – a set of valid characters to be used to verify the correctness of sequence data,
- method `__len__()` – returns the length of the sequence data,
- method `__str__()` – returns a representation in FASTA-like format (see above),
- method `mutate(position, value)` – modifies sequence data at the given `position` to the given `value`,
- method `find_motif(motif)` – returns the positions of the given `motif` in the sequence,
- method `complement()` – returns the complementary sequence,
- method `translate()` – returns a `ProteinSequence` instance representing the result of the translation of the sequence.

In `ProteinSequence`:

- attribute `identifier`,
- attribute `data` – actual sequence,
- attribute `valid_chars` – a set of valid characters to be used to verify the correctness of sequence data,
- method `__len__()` – returns the length of the sequence data,
- method `__str__()` – returns representation in FASTA-like format (see above),
- method `mutate(position, value)` – modifies sequence data at the given `position` to the given `value`,
- method `find_motif(motif)` – returns the positions of the given `motif` in the sequence.

The above specification obviously calls for creating some abstractions and generalizations. This is up to you to decide whether to use composition, inheritance, or inheritance with abstract classes, or a mix of them. However, be prepared to justify your design choices. We also encourage you to try more than one option!

It is also up to you to decide whether an attribute is an instance or class attribute and how to control access to attributes (and methods). Hint: familiarize yourself with the Python convention for leading underscore(s) and with `@property` and `@name.setter` decorators.